# Marigold: Transparent Bearer Notes on a Fully Auditable Chain

The Marigold Project · marigold.cash · note@marigold.cash

v0.6, October 2026 · testnet edition

## Contents

**Abstract** 2

**1. Motivation** 2

**2. Transparent bearer notes** 3

**3. Value conservation: the whole system in one equation** 5

**4. Wallets without seed phrases** 6

**5. Consensus integration: native operations, not contracts** 7

**6. What the concept needs from a chain — and the chain chosen** 8

**7. What an observer sees — the honest section** 8

**8. Economics** 10

**9. Finality anchors: a small safeguard that stays** 10

**10. Related work and future directions** 11

**11. Conclusion** 12

**Status: testnet edition.** This paper describes the network as it runs on the public testnet. It is revised as the testnet teaches us. The normative specification of everything described here is POOL-SPEC.md (v1.3; v1.1 was externally reviewed); where this paper and the specification disagree, the specification is correct. This paper is the readable account: what transparent bearer notes are, why they exist, and what they do and do not provide. Since v0.4: locked handovers (Sections 2, 5 and 7), share keys and receipts (Sections 2 and 4), the owner's lane (Section 8), and the anchors running live (Section 9). The conservation rule of Section 3 and the pool commitment are unchanged.

## Abstract

Physical cash has two properties no mainstream digital money reproduces together: it is a bearer instrument — possession is ownership, and handing it over is settlement — and it carries no memory of who held it. Transparent blockchains provide neither: every unit's full custody history is permanently recorded and clusterable. Privacy chains provide the second property by cryptographically hiding recorded data, at the cost of verification machinery few holders can personally check and a regulatory classification that follows the machinery. Marigold takes a third path: instead of hiding the linkage between payer and payee, it **never records one**. Value is carried by fixed-denomination *notes* — plaintext on-chain entries, each a serial number, a denomination, and a public key. Each note is, in effect, a minimal crypto wallet — one key guarding one fixed value — and a payment exchanges the wallet itself rather than moving value between accounts; hence we call them not *wallets* but *notes*. Whoever holds the matching private key holds the note; spending is a transfer of key control, and the chain records only that a note was retired and reissued under a fresh key — never who did it, because "who" is not a concept the protocol contains. Every note, every denomination, and every operation is public; supply is accountable at every block by a consensus-enforced conservation rule. Nothing is hidden: the chain is complete about value by construction, like physical cash. Marigold instantiates this design on a fork of Kaspa's rusty-kaspa node — a 10 blocks-per-second proof-of-work DAG whose sub-second confirmation makes handing over a note feel like handing over cash — with a fixed 210,000,000-coin supply and a fair launch from zero.

## 1. Motivation

Cash works because of three properties working together. It is **final**: handing over a banknote settles the payment, with no intermediary able to reverse or withhold it. It is **fungible**: one note of a denomination is exactly as good as another, because nothing about a note tells you where it has been. And it is **verifiable by its holder**: you can check a banknote is real without trusting the person who gave it to you, and without a PhD.

Digital money keeps failing to combine these. On transparent chains such as Bitcoin, settlement and holder-verifiability are excellent, but fungibility is structurally absent: every coin carries its complete custody graph forever, address clustering is an industry, and coins are routinely discriminated by history. Privacy chains such as Monero and Zcash restore fungibility by *hiding recorded data* — ring signatures, stealth addresses, confidential amounts, zero-knowledge proofs. The data model still contains senders, receivers, and amounts; cryptography obscures them. This works, but at two costs that matter for a cash replacement. First, verification is no longer personal: the holder must trust that a sophisticated cryptographic construction, which very few humans can audit end to end, contains no flaw and no trapdoor — and history shows such constructions have contained both. Second, the category itself has become a regulatory magnet: assets classified as "privacy coins" face delistings and, in the EU from 2027, statutory restrictions — a categorization that attaches to the *hiding machinery*, whatever the intent.

The observation Marigold is built on: **fungibility does not require hiding anything, if the protocol never records the linkage in the first place.** A banknote is not private because its serial number is encrypted — the serial is printed right on it. It is private because the note carries no ledger of hands it passed through. That property can be built digitally, in plaintext, with no cryptography beyond ordinary signatures: let the chain track *notes* rather than *accounts or flows*, let ownership be

knowledge of a key rather than an identity, and let a spend be a key rotation rather than a sender-to-receiver transfer. Then there is no encrypted sender field to attack or regulate — there is no sender field.

There is an underlying realization that actually produced Marigold. The standard architecture of cryptocurrency has two kinds of object: **wallets**, which hold balances, and **transactions**, which move value between them — and what crypto named a "wallet" is not like a traditional leather wallet at all; it is a bank account. Marigold turns this on its head. A Marigold note is, in crypto terms, a *minimal wallet* — one key guarding one fixed balance — and a transaction no longer moves value between accounts: it hands over the note itself, like a classic payment at the market, by changing the one key that controls it. Value never moves; notes do. Seen this way, a note's private key plays the role a seed phrase plays for a conventional crypto wallet — except that it guards exactly one coin, so possessing, sharing, or losing a key has the scope of a single banknote, never of a life's savings. The rest of the system then follows almost mechanically: notes whose balances never change must come in fixed denominations, with change-making (split and merge) in place of arbitrary amounts; a transaction must be as simple as handing over a key; and the software a user installs stops being a crypto "wallet" at all — it is a keychain for the notes they hold (Section 4), far closer to a traditional leather wallet.

We call the result a **transparent bearer-note chain**. It is not a privacy coin, and this paper will not claim privacy properties the design does not have (Section 7 states exactly what an observer sees). What it claims is narrower and verifiable by anyone: value is fully accounted for at every block, nothing on the chain is encrypted or obfuscated, and the ledger simply does not contain the payer→payee relation — the same way a cash economy's ledger doesn't.

## 2. Transparent bearer notes

Marigold's chain has two kinds of value. The **ledger** is the ordinary account side inherited from its host chain — UTXO entries holding arbitrary amounts, spendable by script signatures, exactly as on Bitcoin or Kaspa. Mining rewards arrive here; exchanges and auditors interact here. Alongside it lives the **note pool** — the currency proper: a consensus-maintained registry of notes.

A note is a triple, entirely public:

```
(serial, denomination, public_key)
```

- The **serial** uniquely identifies the note for its whole life — which, like a UTXO's, runs from the operation that creates it to the operation that consumes it. It is derived, not assigned: `serial = H(creating_txid || output_index)`, a domain-separated hash — the same outpoint pattern the host chain already uses for UTXOs, requiring no counter and no registry of "next serial." Every operation that touches a note retires its serial permanently and issues successors with fresh serials derived from that operation's own transaction; the pool map only ever inserts and removes entries, never mutates one. (This is also why replaying an executed operation is structurally impossible: the serial it would consume no longer exists.)
- The **denomination** is one of eight fixed values: **0.01, 0.1, 1, 10, 100, 1,000, 10,000, 100,000 MAGLD**. Notes have no other amounts, ever. Fixed denominations are what make notes interchangeable — nothing about a 10-note *itself* distinguishes it from any other 10-note, exactly as with a €10 bill; what its operational history may reveal is Section 7's subject.
- The **public key** is the note's current lock. Whoever knows the matching private key owns the note — full stop. There is no account above it, no identity behind it, and no registration

linking it to any other note.

Six operations, and only six, change the pool:

| Operation | Consumes | Produces | Bridges to ledger? |
|---|---|---|---|
| **Mint** | ledger funds | new notes | yes (in) |
| **Rotate** (transfer) | a note (authorized by its current key) | a successor note: fresh serial, recipient's key | no |
| **Rotate, locked** | a note (authorized by its current key) | a successor note under the receiver's key, carrying a deadline and the payer's refund key | no |
| **Split** | one note | ten notes of 1/10 the value | no |
| **Merge** | ten equal notes | one note of 10× the value | no |
| **Redeem** | notes | ledger funds | yes (out) |

Spending a note means transferring control of its key — in the simplest case literally giving the private key to the recipient, by QR code, by NFC tap, by reading it aloud — after which the recipient immediately **rotates**: signs, with the received key, a request installing a fresh public key only they control. The settlement rule is universal and simple enough to print on a card: *a note is finally yours when a rotation to a key only you know is confirmed on-chain.* Three handover styles are supported as equals:

**Bearer handover.** The key itself changes hands, then the recipient rotates — passive receipt, the "granny at a market" flow, for every situation where the recipient cannot or does not issue a payment request first. It has one security property that must be stated plainly: from the moment the key is shown until the recipient's rotation confirms, payer and payee both know it, and whoever rotates first wins. This is the digital analog of the moment cash changes hands — the same exposure, and the same answer: complete the handover immediately, which sub-second confirmation makes practical rather than aspirational. No protocol timeout can close this window, because the payer's residual power is *knowledge of a key*, and knowledge cannot be made to expire.

**Sign-to-fresh-key.** The recipient supplies a fresh public key in a payment request, and the payer's wallet signs the rotation directly to it — the private key never travels, the point-of-sale flow. This removes the shared moment entirely, and it is the natural choice whenever the parties can exchange a payment request; bearer handover exists for the situations where they cannot.

**Locked handover.** The payer rotates the notes to a key only the receiver holds and puts a deadline on them. Until the deadline the notes answer to the receiver's key alone; from the deadline on, to the payer's refund key alone. The receiver takes the money whenever they like inside the window, knowing nobody else can; if they never do, it is the payer's again, with nobody's cooperation required. This is the flow for paying someone who is not there to receive: an invoice settled overnight, a deposit, a payment to a shop's standing key. The condition lives on the note and is enforced by consensus, because nothing done in a code alone can refuse the payer's own key for a while.

The first two styles reduce to the identical on-chain operation and differ only in the shared-knowledge window above; the third is its own operation, the locked rotation of Section 5. Paying a request also hands the payer a **receipt**: the transaction id, the request it answers and the amount, which the receiver's wallet checks against its own node. A receipt is a pointer to a public event, not a proof in itself; the proof is the rotation on the chain.

Split and merge move along the ×10 ladder, so any amount is payable in a handful of notes, the way any cash amount is payable in a handful of bills — and, as with bills, making change is part of paying.

## 3. Value conservation: the whole system in one equation

Every transaction touching the pool must satisfy a single rule, enforced by consensus:

```
Σ(notes consumed) + Σ(ledger inputs) = Σ(notes produced) + Σ(ledger outputs) + fee
```

Everything else follows from it. A mint consumes ledger funds and produces notes of equal value. A redeem consumes notes and produces ledger funds. A rotate consumes a note and produces its equal-denomination successor. A split's ten outputs sum exactly to its input. And **fees are the gap**: any shortfall of produced value against consumed value is the transaction fee, credited to the including block's miner through the host chain's native value-in-minus-value-out accounting — no new payment machinery at all.

This yields Marigold's characteristically boring fee mechanism, the **fee stamp**: to pay the fee on a pure pool operation (which touches no ledger funds by design), the payer simply attaches one small note to the consumed set and produces nothing for it. The stamp's *note* ceases to exist; its *value* does not — it crosses to the ledger inside the miner's block-reward payout, and the conservation equation holds across the whole event. **Nothing in this system is ever burned**: heavy pool usage does not shrink supply, it circulates value from note holders to miners, who (or whoever they pay) mint it back into notes at will. Nor can any denomination become scarce — notes are not a printed stock but are created on demand by mint and split. No wallet ever needs to hold ledger funds or an address to use the pool — the wallet holds nothing but note keys. Congestion pricing works natively: attaching more or larger stamps raises the transaction's fee density in the existing mempool ordering, a denomination-quantized fee market. Mint and redeem, which touch the ledger anyway, pay ordinary fees from it with petal-fine granularity and need no stamp.

> *In one line, because it is the easiest thing in this paper to misread: a fee stamp destroys the **note**, never the **value**. The note vanishes from the pool; its value reaches the miner inside the block reward, and can be minted into new notes at any time.*

The conservation rule also powers the system's headline audit property. Every block header carries a **pool commitment** — a 32-byte cryptographic commitment (a sparse-Merkle-tree root) to the entire note pool's state, the pool-side twin of the host chain's UTXO commitment. Every node verifies, at every block it validates, that:

```
Σ(all notes in the pool) + Σ(ledger supply) = total emitted supply
```

Supply is publicly accountable at every block, by consensus — not by trusting an auditor, and not modulo the soundness of a proof system. A divergent node visibly forks off. This is the precise, checkable content behind the phrase *fully auditable chain*.

## 4. Wallets without seed phrases

A Marigold wallet is not an account. It is a **key ring**: software managing a collection of independent note keys, none derived from any master secret. This is Section 1's inversion made literal: the notes are the wallets; the app is only the keychain. Its jobs are correspondingly cash-like: receive keys (QR/text), rotate immediately, plan payments (choosing and splitting notes to cover an amount, like making change), and keep keys safe.

For keys that arrive from other people, independence is forced: receiving a bearer note means receiving a private key someone else's wallet generated, and no seed phrase of yours can regenerate a key you did not create. Immediate rotation does not empty this category — at any moment a wallet may hold a just-received note, a payment accepted offline and not yet rotated, or a bearer key deliberately kept as-is, like the gift envelope below.

For the fresh keys the wallet generates itself — rotations, mints — independence is a choice. Those keys *could* hang off a master seed; the received keys above never could, so a seed would split custody into two stories — some notes recoverable from words alone, others only from the wallet's files — and a recovery story with exceptions is how users lose money. It would also quietly rebuild the one secret with a life's-savings blast radius that the bearer model exists to remove (Section 1). So every note key is an independent secret, whatever its origin, and recovery is unified by encryption rather than derivation.

One thing does still need a conventional seed: the wallet touches the ledger — exchange deposits and withdrawals, collecting mining rewards — and the ledger side keeps its ordinary seed phrase. Both kinds of key live in one place: the wallet's **note store**, one encrypted file per note plus an encrypted record of the ledger seed, protected by a single 24-word recovery secret. The 24 words are chosen for familiarity, not function: they are an encryption key, not a seed — they derive nothing and generate no keys.

Recovery therefore always requires **your files and your key, period** — and this is deliberate, not incidental. A conventional seed phrase *is* the keys: whoever holds it holds everything it ever derived. Here, the word list alone reconstructs nothing, and the files alone are ciphertext; neither component alone grants anything at all.

One procedure covers every recovery: restore the files, enter the words, decrypt. Restoring rotates every note to fresh keys by default, on the assumption that any backup copy may have been read. The ledger seed cannot be rotated in that sense — it is a derivation mechanism, not a bearer key — so the standing guidance is to keep on the ledger only what is actively in transit to or from an exchange.

Because note keys are independent secrets, nothing ties them to one application. You can run several wallets, each with its own notes, and move notes freely between your own wallets — export from one, import into the other, no on-chain operation, no fee: ownership is knowledge of a key, and the knowledge merely changed pockets. (Handing a key to *someone else* is, of course, just bearer handover, and the settlement rule applies in full.)

Backups are consequently ordinary file backups — an encrypted archive of the wallet's files, kept on your own cloud storage, a USB stick, or posted by the wallet itself into its owner's Telegram chat, where it keeps the copy current without being asked. A backup covers every wallet on the computer, and each wallet's part is sealed to a public key its own 24 words name, so each opens with its own words and never with a wallet password — a reserve wallet that is never opened is

backed up all the same. The distinction matters because a backup lives on servers you do not control, where it can be attacked offline and at leisure: a password is typed every day and chosen to be remembered, which is to say it is not random; the words are 256 bits nobody chose. There is also a printable **paper-QR export** for cold storage. The paper export is deliberately self-contained: its pages are encrypted under their own short password, freshly generated and printed once at export time — recovering from paper needs the pages and that password, and never the wallet's files or the 24 words. A safe, a drawer, an inheritance envelope: the paper works even if every device and every wallet file is gone. Backups deserve the same bluntness as everything else here: a backup is a copy of the keys, so whoever holds both the wallet's files and the words — or a paper page and its password — holds the notes. Recovery is a custody convenience built on that fact, never an exception to bearer ownership.

A **share key** is the one standing key in the design: a public key a receiver publishes once — on a card, in a shop window, in a profile — from which each payer derives a fresh one-time key for the notes they send, using an ephemeral key carried in the payment code. The receiver's wallet recovers each one-time secret from the share secret; the chain sees unrelated keys, one per payment, and nothing that links them to the share key or to each other. Share keys are what locked handovers are paid to, and what lets a wallet be paid while nobody is watching it.

Two further flows round out the cash analogy. A **point-of-sale landing pad** — an implemented wallet mode, not a proposal — lets a merchant device receive payments to fresh single-use keys and sweep them, without the terminal ever holding long-term value; the name means exactly what it says: keys where value only ever lands and departs. And because notes are just keys, **gifting, inheritance, and escrow** are physically representable: a printed QR in an envelope *is* the money, with the same properties — and the same responsibilities — as an envelope of cash.

## 5. Consensus integration: native operations, not contracts

The note pool is implemented as a first-class consensus feature — six operation types with fixed wire formats, validated by every node — rather than as a smart contract. This is a deliberate simplicity choice with three consequences. The action space is **enumerable**: everything that can ever happen to a note is one of six operations whose complete validation rules fit in a short specification, which is what made external review of the entire design tractable. There is **no execution environment** to escape, meter, or misprice. And operations commute the way the host DAG needs them to — parallel blocks containing pool operations merge under the same deterministic rules as parallel blocks containing ordinary transactions.

Pool-operation authorization is a BIP340 Schnorr signature by the note's current key over a domain-separated hash that binds the operation, its consumed serials, the enclosing transaction's ledger outputs (preventing redirect malleability), and a **freshness anchor** — a recent DAA score after which the signature expires (about one hour). Expiry bounds the shelf life of any signed-but-unbroadcast operation: a photographed payment QR or a stale invoice stops being a live liability within the session that created it, closing the class of replay problems bearer instruments are historically prone to. Replay of *executed* operations is impossible by construction: every operation permanently retires the serials it consumed (successors carry fresh serials, even when a key is reused), so a replayed authorization refers to notes that no longer exist.

The locked rotation is the sixth operation, added after the specification's external review. A lock is two fields on a produced note: a refund key and a deadline, expressed as a DAA score. Validation

asks one further question when the note is next spent: if the point-of-view DAA score is at or past the deadline, the authorizing signature must be the refund key's; otherwise the note's own. The rule is monotone — a lapsed lock never un-lapses — and it is evaluated against the same point-of-view score as the freshness anchor. An unlocked note keeps its commitment leaf and its signature preimage byte for byte, so every commitment and every signature from before the change stands unchanged; a locked note's leaf includes its lock, so a peer cannot serve the note without it. Locked rotations activate at a fork height of their own, before which no node holds a locked note. On the testnet they are live.

## 6. What the concept needs from a chain — and the chain chosen

Nothing in Sections 2–5 is specific to any particular blockchain. Transparent bearer notes could be grafted onto most UTXO chains: the design needs a proof-of-work or otherwise neutral base layer, a UTXO-style parallel-friendly data model, ordinary signature verification, and room for a small set of new operation types. The value proposition of this system is the note pool — not its host. We state this plainly because it defines what Marigold is *for*: it is an instantiation of a portable idea, chosen and tuned for one job, digital cash.

That job dictates the host-chain requirements, and they are unforgiving in one dimension above all: **confirmation latency is handover latency.** The settlement rule — yours when your rotation confirms — means the time-to-first-confirmation is the time two people stand at a market stall waiting. On a 10-minute-block chain, bearer handover is unusable; on a 15-second chain, awkward; below one second, it feels like handing over a bill.

Marigold is therefore built as a fork of **rusty-kaspa**, the Rust node of the Kaspa network: a GHOSTDAG proof-of-work blockDAG producing **10 blocks per second**, giving sub-second first confirmation and rapid deepening finality, with mature pruning (the pool's state-commitment design means pruned nodes retain full verification power over current state), a production-quality P2P/RPC stack, and an actively maintained upstream whose fixes Marigold deliberately keeps mergeable. We credit the Kaspa project plainly: Marigold's base layer is their engineering, kept intact precisely because it is excellent. Marigold changes network identity and economics, adds the note pool, and changes nothing else — the fork discipline is "smallest possible diff against upstream," both for security and for honesty about where the base layer's credit belongs.

## 7. What an observer sees — the honest section

This section makes the claims exact, because a cash-like system must be honest about its envelope. **Everything on the chain is public and unencrypted — and "public" means two distinct things here, worth separating.** The pool's *current state* — every live serial, its denomination, its current public key, the pool's total composition per denomination — is plaintext held by every full node and queryable at any moment. A note's *operational history* — when it was minted, every rotation, split, and merge, when it was redeemed — is public in a different sense: each operation was broadcast to the entire network in the block that carried it, and anyone was free to record it, but ordinary nodes prune old block data and retain none of it, so reconstructing a note's past requires an observer who kept the history — an archival node, which can withhold history but never falsify it: the per-block commitments authenticate whatever history is served, and anyone may keep their own copy. Pruning is a storage optimization, not an eraser: it changes *who holds* the history, never whether it was disclosed — and the only safe assumption for anyone relying on this system is that **at least**

**one archival observer always exists.** Amounts are never confidential in either sense, and mint and redeem visibly connect ledger funds to specific serials at entry and exit.

What the chain does **not** contain is any identity attached to any of it. A rotation is a signature by a key over a new key; the protocol has no sender, no receiver, no address book, no account. Unlinkability between the parties of a payment is therefore not an added feature but an absence of recorded data — and the following limits apply, stated without minimization:

- **The operation graph is visible.** Every operation publicly lists the serials it retires and the fresh serials it creates, so a note's custody timeline is a public chain of transactions — rotation replaces the serial, but the rotating transaction itself links predecessor to successor. One precise qualification: operations link *sets* to *sets* — a transaction consuming twenty notes and producing twenty records no pairing of which old note "became" which new one; only a plain one-to-one rotation links exactly. Timing correlations, denomination patterns, and fee-stamp lineage remain analyzable graph structure throughout.
- **The anonymity set of a note is, at best, its denomination cohort** — a 100-note hides among 100-notes, nothing more. Behavioral patterns are the sharpest edge of this limit: several same-denomination rotations in quick succession, a split followed by immediate spending of its pieces, or a merchant's rhythmic accept-then-merge cycle all read as single-entity signatures in the public graph. Wallet hygiene (rotating on receipt, not reusing keys, sensible stamp selection) preserves the baseline, and wallets can blunt the patterns — jittered sweep timing, batched operations, varied payment denominations — but pattern discipline is mitigation, not cryptography, and this paper claims nothing stronger.
- **A locked note shows its terms.** Its pool entry carries the deadline and the refund key beside the receiving key, and both are public for as long as the lock stands; the notes it rotates into are plain. What is on the chain is a deadline and two keys. Who holds either key is, as everywhere else on the chain, not recorded.
- **Pool entry and exit are the transparent seams.** Whoever links a ledger address to an identity learns which serials that identity minted or redeemed. Between those seams, the chain records no linkage; at them, it visibly does.
- **One ledger address per account, and change returns to it — by design, not omission.** Anyone who inherits Kaspa tooling expects a wallet to rotate change addresses and will read their absence as a bug. Here the ledger is the observable perimeter on purpose: it is where exchanges, auditors and the conservation check look, and rotating addresses on a fully public side buys little (change heuristics relink them anyway) while costing gap-limit scans and probabilistic recovery. Privacy is the note's job; value that wants it leaves the ledger and becomes a note, where a fresh key per payment is the default. Restoring an account from its words means deriving one key and asking about one address, with nothing to miss.
- **A note for wallet builders.** Fresh key per rotation is the default for a reason: key reuse is protocol-permitted, but it collapses the anonymity set of every note involved, and systematically biased fee-stamp selection writes a signature into the graph. The specification records these as mitigations, not consensus rules — the protocol will not stop a careless wallet, so the wallet must not be careless.

The regulatory posture follows from the mechanism, not from marketing: exchanges and auditors interact exclusively with the transparent ledger; every pool entry and exit is a visible, value-conserving public event; and nothing on the chain is hidden from anyone. Marigold declines the "privacy coin" label as a matter of accuracy: coins in that category cryptographically conceal recorded data, and this system conceals nothing — the distinction is architectural, not euphemistic.

What it shares with cash is narrower: the chain is complete about *value* by construction, and silent about *people* by construction.

## 8. Economics

**Supply.** Hard cap of **210,000,000 MAGLD**, no tail emission, no premine, no development fund, no allocation of any kind: every coin enters circulation through mining from block zero (a fair launch, announced publicly in advance with binaries available to all). The base unit is the **petal**: 1 MAGLD = $10^8$ petals.

**Emission.** Smooth geometric decay from genesis: the block reward declines in monthly steps of factor 2^(−1/36) — a halving every three years with no cliff moments — from an initial ≈1.5228 MAGLD per second. Roughly 20.6% of supply is emitted in year one and ~90% by year ten; the per-block reward quantizes to its 1-petal floor around year 72. A permanent consensus test verifies the emission table against the cap: it can never exceed it, and monthly rounding leaves the realized total a negligible sliver — under 0.004 MAGLD — beneath it.

**Fees and the long game.** All fees — ledger fees and destroyed fee stamps alike — go to miners, never burned and never to any fund. The security budget is designed around the system's nature as a *circulation* coin: every payment is an on-chain rotation paying a fee, so a successful cash economy is a durable fee base in a way that store-of-value settlement is not. The stated security posture across time: launch guards protect youth (Section 9), emission carries the middle decades, circulation fees carry maturity. That is the bet, made openly.

**Smaller notes when the time comes.** The smallest note is 0.01 MAGLD and the fee is one such note. Should 1 MAGLD come to be worth more than about a dollar, that is no longer small change, and the plan on record is to add a 0.001 note (or smaller) and make the fee the new smallest note. The denomination table has room reserved for it (tags 8–255 in POOL-SPEC), adding a tag is a hard fork by construction — every node and wallet learns the new rung — and nothing else changes: every existing note and denomination stays exactly as it is. The ladder does not start at a petal because every denomination is a separate note that has to be keyed and rotated; a payment spread across nine sizes touches many more notes and fee stamps than one across four, which is why cash stops at the cent.

**The owner's lane.** A wallet's housekeeping — merging small notes, rotating after a restore, minting mining rewards into notes — is traffic the owner generates for themselves. A node run by that owner, on the owner's machine with a miner behind it, accepts that traffic at a nominal fee of one petal per gram, below the relay floor, and keeps it out of relay: no other node carries it and no other miner is asked to mine it. This is node policy, not consensus; against anyone else's node the wallet pays the network rate, and an own-lane transaction not mined within the hour is repriced to the network rate by the wallet itself. The relay floor and the fee market for everyone else are untouched.

## 9. Finality anchors: a small safeguard that stays

A young proof-of-work chain's hashrate is small, and small hashrate invites deep-reorg attacks. Marigold launches with an explicit and fully disclosed guard: **finality anchors**. A set of five publicly identified trustee keys, held by operationally independent parties, periodically co-sign (threshold

3-of-5) an attestation pinning a recent block. Consensus treats a validly anchored block as irreversible: no reorg may cross it. Anchors make deep reorganizations impossible while hashrate grows, at the cost of a disclosed trust assumption — and the design constrains that assumption tightly. Trustees produce no blocks, receive no rewards, and cannot censor, mint, or move anyone's funds; their only power is to *veto reorgs*, and the mechanism **fails open**: if trustees fall silent, the chain continues as ordinary proof of work, degraded not halted. Equivocation by a trustee key (anchoring conflicting branches) is a consensus-visible offense that permanently disqualifies the key.

Earlier editions of this paper described the anchors as temporary, retiring on a difficulty schedule. That is no longer the design, for a reason that arrived with the wallet itself. Most Marigold wallets run no node of their own: a wallet takes the latest anchor from a public node, checks the trustees' signatures over the anchored block, checks the note-pool state it is handed against that block header's pool commitment, finds its own notes in it without naming them, and follows the chain from there, checking every later anchor against what it has followed. Such a wallet verifies no proof of work. The anchor is the only thing it can check the public node against — and an anchor that consensus no longer enforced would be worse than none, since a false one would then fool the wallets without splitting the full nodes that would expose it. So the anchors stay mandatory for as long as the network runs, and enforcement is what makes a false anchor visible to everyone.

The trust is therefore stated without a time limit, and bounded instead by who holds the keys and how they are replaced. Three of the five keys, compromised or colluding, can sign a false anchor; with one cooperating node they can show a wallet without a node a pool state that is not the network's. A full node is unaffected, and anyone who wants no trustee in the loop runs one. Trustee keys are replaceable by the documented key ceremony; succession without a hard fork — a trustee-set update signed by the current quorum, taking effect after a delay every node and wallet can see — is a committed design item, and the intended trajectory hands the *selection* of successors to note-weighted governance (Section 10). The founders choose the first set, not the last. Succession failure, like trustee silence, degrades the chain to plain proof of work; nothing halts.

On the public testnet the guard runs as written: five trustee keys, four signers on two hosts kept apart from the largest miner, an anchor every 300 DAA scores, every node enforcing, every light wallet starting from the latest anchor, and the fail-open path exercised whenever the signers are restarted. The testnet trustees are throwaway keys made for the rehearsal; the mainnet set is chosen at launch. A deliberate deep reorganisation attempted from the largest miner against the anchored chain was rehearsed on 2026-09-22 and refused by every node.

## 10. Related work and future directions

**Chaumian e-cash** (Chaum, 1982) is the intellectual ancestor: digital bearer tokens with genuine unlinkability. Its structural limit was the mint — a trusted issuer who could inflate invisibly or vanish. Marigold keeps the bearer-instrument model and replaces the mint with a public consensus: issuance is the disclosed emission schedule, and the "mint's books" are the conservation rule every node checks. **Monero and Zcash** solve fungibility by encrypting or obscuring a ledger that still records the payer→payee relation; Marigold removes the relation and encrypts nothing — weaker concealment, radically simpler verification, different regulatory shape. **Transparent chains** offer the same verifiability with no fungibility; Marigold is what a transparent chain looks like when the data model itself is redesigned around value instead of flows. **Physical cash** remains the benchmark: Marigold's design goal throughout is to be describable to a cash user in one sitting,

mechanism included.

Future work, deliberately outside launch scope, includes **note-weighted governance**: proposals and votes as native pool operations, where a vote is a signature by a note's current key, weighted by denomination, one vote per serial — sybil resistance from coin weight and double-vote prevention from serial uniqueness, using only machinery already in the system. Its first mandate would be electing trustee successors, so that the anchors pass from the founders' choice to the holders' (Section 9).

## 11. Conclusion

Marigold is a small idea taken seriously: that digital cash fails not for lack of cryptographic sophistication but from an excess of it — and that the fungibility cash users actually rely on comes from a ledger that never wrote the compromising fact down, not from one that hides it well. The system presented here is deliberately legible end to end: five operations, one conservation equation, plaintext state, ordinary signatures, a supply audited by every node at every block, hosted on a base layer fast enough that handing over a note feels like handing over money. Everything it does can be checked by the people it is for. Nothing is hidden — the chain is complete about value by construction, like physical cash.

---

*Specification: POOL-SPEC.md (v1.1, externally reviewed). Source: github.com/marigoldcash/marigold. Contact: security@marigold.cash (security), note@marigold.cash (general).*